Journal of Cloud Computing
https://doi.org/10.1186/s13677-026-00894-6
Article in Press

# Data-intensive task scheduling in geo-distributed cloud computing: a deep reinforcement learning approach

Received: 22 November 2025
Accepted: 23 March 2026
Published online: 13 April 2026
Cite this article as: Liu Z., Sun Y., Liu C. *et al.* Data-intensive task scheduling in geo-distributed cloud computing: a deep reinforcement learning approach. *J Cloud Comp* (2026). https://doi.org/10.1186/s13677-026-00894-6

Zhaoze Liu, Yajuan Sun, Cong Liu, Zhiming Zhao & Long Cheng

We are providing an unedited version of this manuscript to give early access to its findings. Before final publication, the manuscript will undergo further editing. Please note there may be errors present which affect the content, and all legal disclaimers apply.

If this paper is publishing under a Transparent Peer Review model then Peer Review reports will publish with the final article.

ARTICLE IN PRESS

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

ARTICLE IN PRESS

# Data-Intensive Task Scheduling in Geo-Distributed Cloud Computing: A Deep Reinforcement Learning Approach

Zhaoze Liu[1], Yajuan Sun[2*], Cong Liu[3], Zhiming Zhao[4], Long Cheng[1]

[1]State Key Laboratory of Alternate Electrical Power System with Renewable Energy Sources, North China Electric Power University, Beijing, China.
[2*]Network and Information Office, North China Electric Power University, Beijing, China.
[3]NOVA Information Management School, Nova University of Lisbon, Lisbon, Portugal.
[4]Research Group of Multi-Scale Networked Systems, University of Amsterdam, Amsterdam, Netherlands.

*Corresponding author(s). E-mail(s): syj@ncepu.edu.cn;
Contributing authors: zhaozezz@ncepu.edu.cn; cliu@novaims.unl.pt; z.zhao@uva.nl; lcheng@ncepu.edu.cn;

ARTICLE IN PRESS

**Abstract**

Geo-distributed clouds have become an increasingly common architecture in modern cloud computing. By placing data centers in multiple geographical regions, cloud service providers shorten the distance between users and cloud resources, enabling low-latency service delivery worldwide. However, this architecture raises new challenges for scheduling data-intensive tasks, including limited inter-regional bandwidth and high traffic costs charged by Internet service providers. These factors can significantly impact both performance and cost efficiency. Existing scheduling approaches for geo-distributed clouds often rely on mathematical and meta-heuristic methods. They typically focus on batch tasks, unable to make real-time decisions and exhibit limited adaptability to changing workload patterns. To address these constraints, this paper introduces a deep reinforcement learning-based framework for scheduling data-intensive tasks across geographically distributed data centers. The framework performs real-time

ARTICLE IN PRESS

scheduling while considering bandwidth constraints and traffic costs, enabling multi-objective optimization of task response time and total operational cost. Extensive experiments under various workloads show that the proposed method achieves competitive and often better performance than the evaluated baselines.

**Keywords:** Geo-Distributed Cloud Computing, Data-Intensive Task Scheduling, Deep Reinforcement Learning

# 1 Introduction

Cloud computing has become a fundamental paradigm for delivering computing resources and services over the Internet. As user bases continue to expand globally, cloud service providers (CSPs) face increasing demands to deliver low-latency and highly available services to geographically distributed users. To meet these requirements, geo-distributed clouds have become an increasingly common architecture in modern cloud computing. By placing data centers in multiple geographical regions, CSPs shorten the distance between users and cloud resources, thereby enabling low-latency service delivery [1]. In addition, users can deploy their applications in multiple regions to enhance fault tolerance and service reliability [2].

Despite the aforementioned advantages, geo-distributed clouds still face significant challenges. Specifically, data transfers between geo-distributed data centers often rely on wide area networks (WANs) with limited bandwidth and high traffic costs charged by Internet service providers (ISPs) [3]. At the same time, data-intensive tasks such as scientific computing and the training and inference of AI models are increasingly prevalent in cloud environments [4, 5]. Their execution often requires transmitting large volumes of data to data centers. In such scenarios, WAN bandwidth limitations and associated traffic costs can not only lead to performance degradation but also increase network expenses for CSPs. Consequently, efficient scheduling methods for data-intensive tasks in geo-distributed clouds are needed, so as to mitigate the impact of constrained bandwidth on system performance while alleviating the financial burden of traffic costs on CSPs.

Existing work on task scheduling in geo-distributed cloud environments has predominantly adopted either mathematical optimization or meta-heuristic algorithms [6]. Mathematical optimization-based methods [7] typically rely on a planning horizon with workload and system information available in advance, enabling a static optimization formulation for offline planning. Consequently, such formulations are difficult to apply in dynamic cloud environments where tasks arrive continuously and system conditions evolve over time. Meta-heuristic methods [8], on the other hand, partially address this limitation by grouping arriving tasks into short time slots and scheduling them in batches. However, their batch-based nature makes it hard to provide real-time scheduling decisions for individual tasks, thus limiting their effectiveness in highly dynamic geo-distributed cloud environments.

Deep reinforcement learning (DRL) provides a promising framework for making decisions in complex and dynamic environments. In this framework, an agent can

ARTICLE IN PRESS

learn a policy that maps states to optimal actions through trial-and-error interactions with the environment [9]. By formulating task scheduling as a sequential decision-making problem, DRL enables real-time scheduling decisions that adapt to changing workloads and cloud system states. Recent studies [10] have demonstrated that DRL has achieved remarkable success in resource allocation and task scheduling in single-region cloud computing systems. Motivated by these advantages, this paper proposes a novel DRL-based framework for scheduling data-intensive tasks in geo-distributed clouds, aiming to overcome the limitations of existing mathematical and meta-heuristic methods. The main contributions of this work can be summarized as follows:

- We propose a holistic scheduling method for data-intensive tasks that enables real-time decision-making across heterogeneous server clusters distributed in multiple regions, while considering inter-regional bandwidth constraints and traffic costs.
- We provide a detailed implementation of our DRL framework, which includes a tailored Markov decision process (MDP) formulation for the scheduling problem and a specialized training algorithm for efficient learning of optimal scheduling policies.
- We perform extensive experiments to compare the performance of the proposed method with existing scheduling approaches. The results indicate that the proposed method adapts well to the evaluated workloads, reducing response time while keeping operational costs competitive.

The rest of this paper is organized as follows: Section 2 reviews existing task scheduling approaches for geo-distributed clouds; Section 3 presents the system model and formulates the optimization objectives; Section 4 describes the DRL-based scheduling framework for data-intensive tasks; Section 5 reports the experimental results; and Section 6 summarizes the paper and outlines future work.

## 2 Related Work

Task scheduling in geo-distributed cloud environments has attracted extensive attention in recent years. Existing studies can be grouped into two main categories: mathematical methods and meta-heuristic methods [6]. Under mathematical methods, the task scheduling problem is typically formulated as an interpretable optimization model and solved with mathematical tools. For example, Yuan *et al.* [11] formulate the problem as a geography-aware convex optimization model and derive the profit-maximizing solution using the interior point method. Li *et al.* [12] model the problem as an assignment problem and employ the Hungarian algorithm to minimize total tardiness under deadline constraints. Luo *et al.* [13] formulate the electricity cost minimization in task scheduling as a mixed-integer nonlinear programming problem and use a submodular-based method to solve it. These approaches typically assume a static set of tasks that is known in advance and generate one-shot scheduling decisions computed offline. Furthermore, they rely on precise analytical models under strict constraints to satisfy the assumptions imposed by the optimization tools. As a result, these mathematical methods often lack scalability and adaptability in highly dynamic and uncertain geo-distributed cloud environments.

ARTICLE IN PRESS

ARTICLE IN PRESS

To improve the scalability and adaptability of task scheduling in geo-distributed clouds, researchers have increasingly turned to meta-heuristic algorithms. These methods simulate natural phenomena to explore large solution spaces to obtain near-optimal scheduling decisions. For instance, Yuan *et al.* [14] employ simulated annealing to address a bi-objective scheduling problem, simultaneously maximizing profit and minimizing the probability of task loss. Ammari *et al.* [15] propose a modified firefly algorithm that reduces the total operational cost of task execution, including ISP bandwidth costs and grid energy expenses. Xie *et al.* [16] design an ant colony-based scheduler to reduce data transfer time and task completion time under constraints on cost, storage capacity, and load levels. To enable continuous scheduling of incoming tasks, these methods typically aggregate tasks into batches within fixed time slots to determine scheduling plans. However, such a batch mode limits their ability to perform fine-grained real-time scheduling of individual tasks. Moreover, the design of solution encodings, fitness functions, and heuristic operators is largely manual and problem-specific, which restricts the adaptability of the resulting algorithms to new workload patterns and system configurations.

DRL has emerged as a powerful paradigm for sequential decision-making in complex cloud systems. The strong fitting capability of deep neural networks has motivated growing interest in applying DRL to real-time task scheduling [17] and resource management [18] within single-region data centers. In the context of geo-distributed cloud computing, Hogade and Pasricha [5] use DRL to schedule AI inference tasks with the goal of jointly reducing carbon emissions and operating costs. Similarly, Zhang *et al.* [19] employ DRL to schedule AIGC workloads among global GPU data centers to optimize GPU utilization, energy expenditure, carbon emissions, and data-migration overheads. These studies confirm the potential of DRL for handling complex and dynamic scheduling problems in geo-distributed clouds. However, they are primarily tailored to specific objectives, such as energy cost and carbon reduction, and specialized workloads like AI inference or AIGC training. In contrast, this paper focuses on real-time scheduling of general data-intensive tasks in geo-distributed cloud environments. We develop a system model that explicitly captures task input-data transfers while accounting for inter-regional bandwidth constraints and ISP traffic pricing. Based on this model, we design a DRL-based scheduler that operates across heterogeneous server clusters deployed in multiple regions, aiming to optimize both task response time and total operational cost.

ARTICLE IN PRESS

# 3 System Model and Problem Formulation

## 3.1 Overview of the System Model

This paper deals with the problem of real-time scheduling of non-preemptive data-intensive tasks in a geo-distributed cloud environment. The system model, shown in Figure 1, consists of three layers: the user layer, the scheduler layer, and the server layer. We assume that the CSP operates data centers in multiple regions, each hosting a collection of heterogeneous servers connected via a WAN. Servers retrieve tasks from their respective waiting queues and execute them in order. In the user layer, users in each region submit task execution requests to the CSP. The scheduler layer

ARTICLE IN PRESS

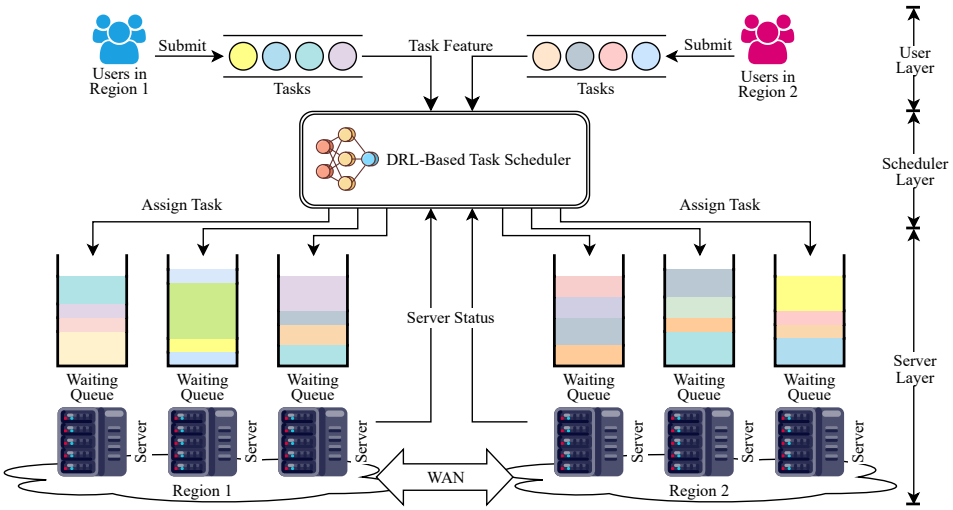


**Fig. 1** The system model for real-time scheduling of non-preemptive data-intensive tasks in a geo-distributed cloud environment.

contains a DRL-based task scheduler that monitors incoming tasks in real time and makes scheduling decisions based on the states of servers across data centers. The task input data are then transferred from the user's region to the server selected by the scheduler. To formulate the optimization problem, we build mathematical models for data-intensive tasks, servers, and the task execution process in the following sections. Table 1 summarizes the notations used throughout the paper.

## 3.2 Data-Intensive Task Model

The scheduler's workload includes data-intensive tasks submitted by users from different regions, denoted by a set $W = \{T_1, T_2, \ldots, T_{|W|}\}$. A task $T_i$ in the set is formally defined as follows:

$$T_i = \{T_i^{\text{arr}}, T_i^{\text{par}}, T_i^{\text{len}}, T_i^{\text{reg}}, T_i^{\text{data}}\} \ (i \in \{1, 2, \ldots, |W|\}) \tag{1}$$

where $T_i^{\text{arr}}$ indicates the task's arrival time, $T_i^{\text{par}}$ is the average parallelism of the task, and $T_i^{\text{len}}$ denotes the baseline execution time. Furthermore, $T_i^{\text{reg}}$ denotes the geographic region of the user who submitted the task. $T_i^{\text{data}}$ represents the input data size required by task $T_i$, which is transferred from the task's submission region to the selected server before execution.

ARTICLE IN PRESS

ARTICLE IN PRESS

**Table 1** Notations used in the system model.

| Notation | Definition |
|---|---|
| $W$ | Workload: the set of tasks to be scheduled |
| $T_i$ | The $i$-th task in the workload |
| $T_i^{\text{arr}}$ | Arrival time of task $i$ |
| $T_i^{\text{par}}$ | Average parallelism of task $i$ |
| $T_i^{\text{len}}$ | Baseline execution time of task $i$ |
| $T_i^{\text{reg}}$ | Submission region of task $i$ |
| $T_i^{\text{data}}$ | Input data size required by task $i$ |
| $M$ | Set of available servers |
| $S_j$ | The $j$-th server in $M$ |
| $S_j^{\text{cpu}}$ | CPU count of server $j$ |
| $S_j^{\text{cost}}$ | Hourly rental cost of server $j$ |
| $S_j^{\text{reg}}$ | Geographic region of server $j$ |
| $S_j^{\text{idle}}$ | Time when server $j$ becomes idle |
| $S_j^{\text{exp}}$ | Expiration time (lease expiry) of server $j$ |
| $R$ | Set of regions where the CSP deploys data centers |
| $b_{ru}$ | Network bandwidth from region $r$ to region $u$ |
| $c_{ru}$ | Per-unit data transfer cost from region $r$ to region $u$ |
| $c_i^{\text{srv}}$ | Server leasing cost incurred by scheduling task $i$ |
| $c_i^{\text{net}}$ | Data transfer cost incurred by scheduling task $i$ |
| $t_i^{\text{start}}$ | Start time of task $i$ |
| $t_i^{\text{end}}$ | Completion time of task $i$ |
| $t_i^{\text{net}}$ | Data transmission time of task $i$ |
| $t_i^{\text{exe}}$ | Actual execution time of task $i$ |
| $t_i^{\text{resp}}$ | Response time of task $i$ |
| $g_i$ | Execution status of task $i$ (1: success; 0: failure) |

## 3.3 Server Model

The servers in all the data centers are represented as a set $M = \{S_1, S_2, \ldots, S_{|M|}\}$. Each element in the set represents a server, defined as:

$$S_j = \{S_j^{\text{cpu}}, S_j^{\text{cost}}, S_j^{\text{reg}}, S_j^{\text{idle}}, S_j^{\text{exp}}\} \ (j \in \{1, 2, \ldots, |M|\}) \tag{2}$$

where $S_j^{\text{cpu}}$ denotes the number of CPUs of the server, $S_j^{\text{cost}}$ is the hourly rental cost of the server, $S_j^{\text{reg}}$ indicates the geographical region of the server, $S_j^{\text{idle}}$ is the time when the server will become idle, and $S_j^{\text{exp}}$ is the expiration time of the server.

## 3.4 Task Execution Model

### 3.4.1 Speedup Model

In order to model the impact of assigning tasks across servers with different numbers of CPUs on the task execution speed, we use the parallel workload model by Downey *et al.* [20] and define the speedup function $SU(n; A, \sigma)$. This function estimates the actual speedup of a task when it is executed on a server with $n$ CPUs, where $A$ is the average parallelism of the task and $\sigma$ is the coefficient of variation in parallelism during

ARTICLE IN PRESS

execution. For the low variance case ($0 \leqslant \sigma \leqslant 1$), the speedup function is as follows:

$$SU(n; A, \sigma) = \begin{cases} \frac{An}{A+\sigma(n-1)/2}, & 1 \leqslant n \leqslant A; \\ \frac{An}{\sigma(A-1/2)+n(1-\sigma/2)}, & A < n \leqslant 2A-1; \\ A, & n > 2A-1. \end{cases} \tag{3}$$

For the high-variance case ($\sigma > 1$), the speedup function is defined as:

$$SU(n; A, \sigma) = \begin{cases} \frac{nA(\sigma+1)}{A+A\sigma-\sigma+n\sigma}, & 1 \leqslant n \leqslant A + A\sigma - \sigma; \\ A, & n > A + A\sigma - \sigma. \end{cases} \tag{4}$$

Figure 2 shows the relationship between speedup and the number of processors for a task with an average parallelism of $A = 16$ for various coefficients of variation $\sigma$. When $\sigma = 0$, the parallelism of a task is constant throughout its execution, and speedup increases linearly with the number of processors (the "hardware limit") until it hits the "software limit" defined by the average parallelism of the task. The increase in $\sigma$ means that the task cannot achieve the expected parallelism during execution, thus leading to a slower speedup scaling with increasing number of processors. When $\sigma \to \infty$, the speedup curve tends toward the lower bound [21]:

$$\lim_{\sigma \to \infty} SU(n; A, \sigma) = SU_{\min}(n; A) = \frac{nA}{A+n-1} \tag{5}$$

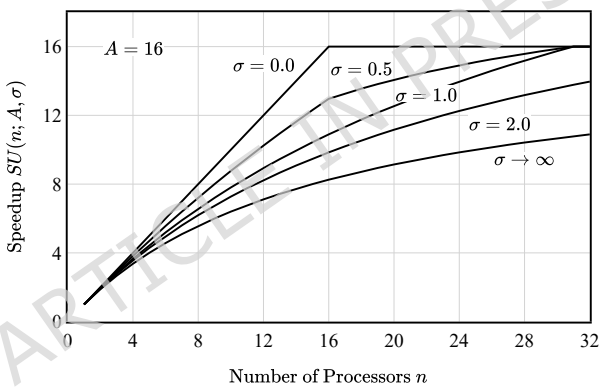


**Fig. 2** Speedup curves for a task with average parallelism $A = 16$ under different coefficients of variation $\sigma$.

### 3.4.2 Task Execution Process

To model the execution of data-intensive tasks on servers, we define the set $R = \{1, 2, \ldots, |R|\}$ to represent all regions in which the CSP deploys data centers. Each element in $R$ corresponds to a region identifier. The matrix $B = [b_{ru}]_{|R|\times|R|}$ represents

ARTICLE IN PRESS

the bandwidth of the network between regions, where $b_{ru}$ is the bandwidth from region $r$ to region $u$. Similarly, the matrix $C = [c_{ru}]_{|R|\times|R|}$ indicates the inter-regional traffic cost, where $c_{ru}$ is the cost per unit of data transferred from region $r$ to region $u$.

Assume that the scheduler assigns task $T_i$ to server $S_j$, where $T_i$ is submitted from region $T_i^{\text{reg}} = r$ and $S_j$ is located in region $S_j^{\text{reg}} = u$. The execution of a task consists of three steps. The first step is server leasing: when $T_i$ arrives, a new lease is initiated for $S_j$ if its current lease has expired; otherwise, no leasing action is performed. The server's expiration time and the corresponding leasing cost are then determined as follows:

$$S_j^{\text{exp}} = T_i^{\text{arr}} + \tau_{\text{rent}}, \quad \text{if } T_i^{\text{arr}} \geqslant S_j^{\text{exp}} \tag{6}$$

$$c_i^{\text{srv}} = \begin{cases} S_j^{\text{cost}} \cdot \tau_{\text{rent}}, & \text{if } T_i^{\text{arr}} \geqslant S_j^{\text{exp}}, \\ 0, & \text{otherwise.} \end{cases} \tag{7}$$

where $\tau_{\text{rent}}$ denotes the lease duration in hours.

The second step is the computation of the start time $t_i^{\text{start}}$ and completion time $t_i^{\text{end}}$ of the task. The start time is given by:

$$t_i^{\text{start}} = \max\left\{T_i^{\text{arr}}, S_j^{\text{idle}}\right\} \tag{8}$$

If the server is busy ($T_i^{\text{arr}} < S_j^{\text{idle}}$), the task waits until the server becomes available; otherwise, execution starts immediately. The input data of task $T_i$ with size $T_i^{\text{data}}$ is transferred from region $r$ to region $u$. The corresponding data transmission time $t_i^{\text{net}}$ and traffic cost $c_i^{\text{net}}$ are calculated as follows:

$$t_i^{\text{net}} = T_i^{\text{data}} / b_{ru}, \tag{9}$$

$$c_i^{\text{net}} = T_i^{\text{data}} \cdot c_{ru} \tag{10}$$

The actual execution time $t_i^{\text{exe}}$ is derived from the baseline execution time $T_i^{\text{len}}$ and adjusted by a coefficient incorporating the speedup function:

$$t_i^{\text{exe}} = T_i^{\text{len}} \cdot \frac{T_i^{\text{par}}}{SU(S_j^{\text{cpu}}; T_i^{\text{par}}, \sigma)} \tag{11}$$

This definition considers both hardware limitations (insufficient CPUs) and software limitations (task parallelism), giving a better representation of the effect of server computational capacity on parallel task execution. The completion time of the task is then given by:

$$t_i^{\text{end}} = t_i^{\text{start}} + t_i^{\text{net}} + t_i^{\text{exe}} \tag{12}$$

The third step is to determine the status of task execution and update the server state. Due to the limitations of leasing, a task can only succeed if it finishes before the server lease expires, otherwise it fails and must be rescheduled. Let $g_i$ be the execution

ARTICLE IN PRESS

status of task $T_i$:

$$g_i = \begin{cases} 1, & \text{if } t_i^{\text{end}} \leqslant S_j^{\text{exp}}, \\ 0, & \text{otherwise.} \end{cases} \tag{13}$$

If the task succeeds ($g_i = 1$), the server's idle time is updated to the task's completion time:

$$S_j^{\text{idle}} = t_i^{\text{end}} \tag{14}$$

Otherwise, both the server idle time and task arrival time are updated to the server expiration time to facilitate rescheduling:

$$S_j^{\text{idle}} = S_j^{\text{exp}} \tag{15}$$

$$T_i^{\text{arr}} = S_j^{\text{exp}} \tag{16}$$

### 3.5 Problem Formulation

In task scheduling problems, response time is an important metric for evaluating scheduling methods. The response time $t_i^{\text{resp}}$ of task $T_i$ is defined as the time between its arrival and completion:

$$t_i^{\text{resp}} = t_i^{\text{end}} - T_i^{\text{arr}} \tag{17}$$

Since users typically want their tasks to complete faster, shorter response times are preferable. For the CSP, the main concern is the operational cost, including the server rental cost $c_i^{\text{srv}}$ in Equation 7 and the data transfer cost $c_i^{\text{net}}$ in Equation 10. The CSP aims to minimize this cost while still delivering an acceptable user experience. In addition, task failures should be avoided as much as possible. A failed task not only wastes the time and cost already spent, but also has a negative impact on user experience. We use the execution status indicator $g_i$ defined in Equation 13 to denote whether $T_i$ completes successfully. Accordingly, for a given workload $W$, this study aims to maximize the task success rate (SR) and minimize the average response time (ART) and total operational cost (TOC). The problem can be formulated as the following multi-objective optimization:

$$\begin{aligned} \text{maximize: SR} &= \frac{1}{|W|} \sum_{i=1}^{|W|} g_i \\ \text{minimize: ART} &= \frac{1}{|W|} \sum_{i=1}^{|W|} t_i^{\text{resp}} \\ \text{minimize: TOC} &= \sum_{i=1}^{|W|} \left( c_i^{\text{srv}} + c_i^{\text{net}} \right) \end{aligned} \tag{18}$$

## 4 The Proposed Method

### 4.1 Preliminaries of DRL

Reinforcement Learning (RL) offers a formal framework for sequential decision-making [22], where an autonomous agent learns to make decisions by interacting with

ARTICLE IN PRESS

an environment over a series of time steps. At each time step $t$, the agent observes the current state $S_t$ of the environment and selects an action $A_t$ according to its policy. In response, the environment provides a numerical reward $R_t$ as feedback and moves to a new state $S_{t+1}$. This iterative agent-environment interaction, shown in Figure 3, allows the agent to learn from the consequences of its actions over time.

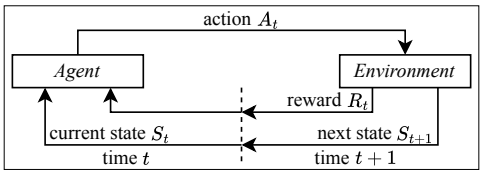


**Fig. 3** The agent-environment interaction in reinforcement learning.

Formally, $S_t$, $A_t$, and $R_t$ are random variables that denote the state, action, and reward at time $t$, respectively. The behavior of the agent is described by a policy $\pi$, where $\pi(a_t \mid s_t) = P\{A_t = a_t \mid S_t = s_t\}$ defines the probability of taking action $a_t$ given state $s_t$. The goal of RL is to find an optimal policy that maximizes the expected cumulative reward. To quantify this objective, the discounted return from time $t$ onwards is defined as:

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots \tag{19}$$

where $\gamma \in [0, 1)$ is the discount factor which controls the relative importance of future rewards. The agent aims to maximize the expected value of $G_t$ by taking actions that lead to high cumulative rewards.

To evaluate the utility of an action taken in a given state, value-based RL methods introduce the action-value function $Q_\pi(s_t, a_t)$, defined as the conditional expectation of the discounted return:

$$Q_\pi(s_t, a_t) = \mathbb{E}\left[G_t \mid S_t = s_t, A_t = a_t\right] \tag{20}$$

The action-value function measures the quality of action $a_t$ in state $s_t$ under policy $\pi$. An optimal policy is one that always chooses actions with the highest achievable discounted return, which is characterized by the optimal action-value function:

$$Q^\star(s_t, a_t) = \max_\pi Q_\pi(s_t, a_t) \tag{21}$$

A representative approach to learning $Q^\star$ is Q-learning [23], which iteratively updates Q-values based on the observed reward and the best estimated value of the next state. In tabular settings, these Q-values are stored explicitly in a Q-table, with each entry corresponding to a specific state-action pair.

However, tabular Q-learning is difficult to scale to complex environments with high-dimensional continuous state spaces. It is impractical to enumerate all state-action pairs and maintain a Q-table in such cases. To address this limitation, the deep

ARTICLE IN PRESS

Q-network (DQN) algorithm [24] approximates the optimal action-value function $Q^\star$ using a deep neural network (DNN) with parameters $\theta$, denoted as $Q(s, a; \theta)$. At time $t$, the agent evaluates candidate actions using the learned DQN and selects the action with the highest Q-value under the current state $s_t$:

$$a_t = \underset{a}{\operatorname{argmax}}\, Q(s_t, a; \theta) \tag{22}$$

By combining Q-learning with a DNN, DQN enables value-based RL to handle high-dimensional decision problems and has been widely adopted across a variety of domains [25].

## 4.2 MDP Formulation

The Markov Decision Process (MDP) provides a formal way of modeling sequential decision-making under uncertainty [26]. It consists of a set of states $\mathcal{S}$, a set of actions $\mathcal{A}$, and a reward function $\mathcal{R}$. We apply this framework to model the problem of data-intensive task scheduling in a geo-distributed cloud as an MDP, so that our DRL agent can learn an optimal scheduling policy by interacting with the environment. In this formulation, the agent observes the current state of the cloud system (including attributes of tasks and servers), chooses an action (assigns a task to a server), and receives a reward based on the quality of the scheduling decision. The components of this MDP are defined as follows.

### 4.2.1 Action Space $\mathcal{A}$

In our MDP formulation, an action is the assignment of an incoming task to a particular server. The action space $\mathcal{A}$ is defined as the set of all the available servers in all data center regions:

$$\mathcal{A} = M = \{S_j \mid j = 1, 2, \ldots, |M|\} \tag{23}$$

For each task $T_i$, the agent chooses a server $S_j \in \mathcal{A}$ for execution. This formulation enables the agent to take into account any server in the geo-distributed cloud as an action possibility, and thus to make globally optimized scheduling decisions instead of limiting the choice to a single region.

### 4.2.2 State Space $\mathcal{S}$

The state space $\mathcal{S}$ contains all the information required for the agent to take informed scheduling decisions. A well-designed representation of the state plays an important role in effective decision-making, because it allows the agent to have a sense of the environment by capturing relevant characteristics of tasks and servers. In our design, there are five groups of features, denoted as $F_1$ to $F_5$:

$$\mathcal{S} = \bigcup_{k=1}^{5} F_k \tag{24}$$

ARTICLE IN PRESS

$F_1$ measures the match between the number of CPUs in each server and the parallelism requirement of task $T_i$. This metric is defined as the ratio of the server's CPU count to the task's parallelism level for all servers:

$$F_1 = \left\{ \frac{S_j^{\text{cpu}}}{T_i^{\text{par}}} \middle| j = 1, 2, \ldots, |M| \right\} \tag{25}$$

For a given server $S_j$, a ratio greater than or equal to 1 means that the server has enough CPU cores to take full advantage of the parallelism of the task, while a ratio less than 1 means that there is not enough CPU resources. A ratio close to 1 is an optimal balance, minimizing both under or over-provisioning of CPU capacity.

$F_2$ is the estimated waiting time for task $T_i$ when it is assigned to each server in the action space. It is defined as:

$$F_2 = \left\{ \max\left(S_j^{\text{idle}} - T_i^{\text{arr}}, 0\right) \middle| j = 1, 2, \ldots, |M| \right\} \tag{26}$$

The expression $\max(S_j^{\text{idle}} - T_i^{\text{arr}}, 0)$ provides a non-negative waiting time for task $T_i$ on server $S_j$ which is zero if the server is idle or becomes available by the time of the task's arrival. The agent can use $F_2$ to assess the load on each server during scheduling, thus minimizing the waiting time of tasks.

$F_3$ indicates the lease status of each server when task $T_i$ arrives and checks if remaining lease time is enough to complete the task. It is defined as:

$$F_3 = \left\{ \frac{S_j^{\text{exp}} - T_i^{\text{arr}}}{T_i^{\text{len}}} \middle| j = 1, 2, \ldots, |M| \right\} \tag{27}$$

A ratio less than or equal to 0 means that the server has expired when the task arrives, requiring a new lease for task execution. A ratio that falls between 0 and 1 indicates that the server is leased when the task arrives, but does not have enough time left to carry out the task. On the other hand, a ratio of 1 or more means that the task can be completed within the remaining lease time when the CPU resources are sufficient. According to $F_3$, the agent can learn to avoid assigning tasks to servers with insufficient lease time and to create a new lease when necessary.

$F_4$ and $F_5$ represent the region information of the incoming task and all candidate servers, respectively. A region $r \in R$ is encoded by a one-hot vector $\boldsymbol{e}(r) \in \{0, 1\}^{|R|}$, where the $u$-th entry is defined as:

$$[\boldsymbol{e}(r)]_u = \begin{cases} 1, & u = r, \\ 0, & u \neq r. \end{cases} \quad (u = 1, 2, \ldots, |R|) \tag{28}$$

Accordingly, $F_4$ encodes the submission region of task $T_i$ and $F_5$ encodes the regions of all servers in set $M$:

$$F_4 = \boldsymbol{e}(T_i^{\text{reg}}) \tag{29}$$

ARTICLE IN PRESS

$$F_5 = \left\{ \boldsymbol{e}\left(S_j^{\text{reg}}\right) \,\middle|\, j = 1, 2, \ldots, |M| \right\} \tag{30}$$

These features can give the agent information about the geographical distribution of tasks and servers, so it can take into account the inter-regional bandwidth limitations and traffic costs in its scheduling decisions.

### 4.2.3 Reward Function $\mathcal{R}$

The reward function measures the result of each action and directs the agent to desirable behavior. A well-designed reward function is crucial to the training of the agent to reach scheduling objectives. To optimize multiple objectives, the reward signal takes both performance and cost into consideration. Specifically, when task $T_i$ is assigned to server $S_j$, the reward is defined as:

$$\mathcal{R} = \begin{cases} R_1 + R_2 + R_3, & g_i = 1, \\ -1, & g_i = 0. \end{cases} \tag{31}$$

$$R_1 = t_i^{\text{exe}} / t_i^{\text{resp}} \quad R_2 = T_i^{\text{len}} / t_i^{\text{exe}} \quad R_3 = e^{\lambda - (c_i^{\text{srv}} + c_i^{\text{net}})}$$

where $g_i = 0$ indicates the failure of the task because of server expiration, so the penalty is $-1$. For successful tasks ($g_i = 1$), the reward has three parts reflecting different scheduling objectives:

$R_1$ is the reciprocal of the response ratio for task $T_i$. It reflects the proportion of waiting time and data transfer time in the total response time. A lower $R_1$ indicates that the task spends much time waiting or transferring data rather than executing. Consequently, maximizing $R_1$ motivates the agent to minimize waiting time and data transfer delays.

$R_2$ is the ratio of the baseline execution time $T_i^{\text{len}}$ to the actual execution time $t_i^{\text{exe}}$ of task $T_i$. Equation 11 indicates that if the CPU resources of the server cannot meet the parallelism requirements of the task, the actual execution time will be longer than the baseline, resulting in a smaller $R_2$. Therefore, maximizing $R_2$ can guide the agent to assign tasks to servers that can meet the parallelism needs and prevent performance degradation caused by insufficient CPU resources.

$R_3$ is the cost-related reward component, which includes the rental and traffic costs incurred by scheduling task $T_i$ on server $S_j$. The sum of these two costs is converted to a positive reward by an exponential function. Lower costs result in values of $R_3$ closer to $e^{\lambda}$, while higher costs lead to values approaching 0. The parameter $\lambda$ controls the weight of the cost factor in the reward. An increase in $\lambda$ emphasizes cost minimization while a decrease prioritizes performance optimization.

## 4.3 Training Process

The proposed DRL-based task scheduler is trained with the DQN algorithm. Figure 4 shows the overall training process. Experience replay is adopted to improve sample efficiency, and a target network is further employed to enhance training stability. Algorithm 1 provides the pseudo-code for the training procedure. During the initialization

ARTICLE IN PRESS

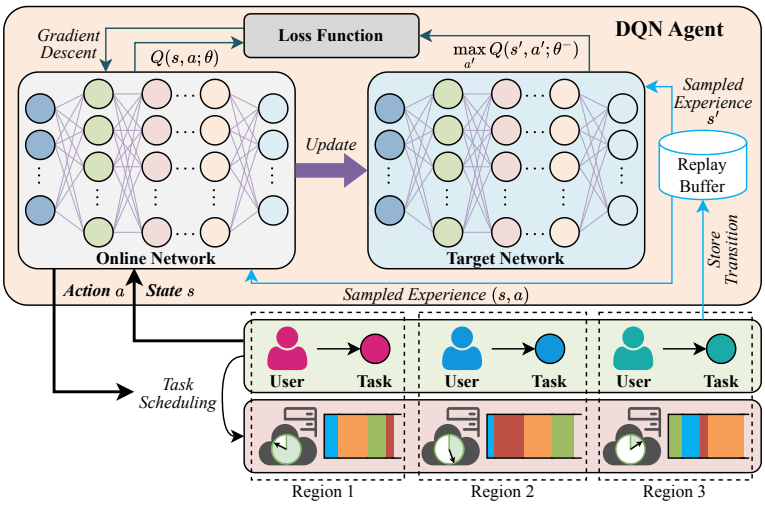


**Fig. 4** The training process of the DRL-based task scheduler.

phase (lines 1-4), the replay buffer, exploration rate and online network are initialized. The parameters of the online network are copied to the target network. As we see in line 5, training is carried out over $N_{\text{episodes}}$ episodes. Lines 8-12 give the operations carried out by the agent for task scheduling. At each decision time step $t$, the agent chooses an action with an $\epsilon$-greedy policy: with probability $\epsilon$, the agent explores by randomly choosing a server and with probability $1-\epsilon$, the agent exploits the DQN by choosing the action $a_t = \text{argmax}_a\, Q(s_t, a; \theta)$ that maximizes the Q-value. The chosen action is then taken, assigning current task to the chosen server, thereby yielding a reward $r_t$ and a new state $s_{t+1}$. This transition $(s_t, a_t, r_t, s_{t+1})$ is stored in the replay memory $\mathcal{D}$ for experience replay. During training, $\epsilon$ is annealed from $\epsilon_{\max}$ to $\epsilon_{\min}$, which helps to make a smooth transition from exploration to exploitation in the agent's behavior.

The temporal difference (TD) algorithm is used to update the DQN in lines 13-21. Once the replay buffer is full enough to create a mini-batch, the DQN starts training on mini-batches of experience. At each training iteration, a mini-batch of size $N_{\text{batch}}$ is randomly sampled from the replay memory $\mathcal{D}$. For each transition $(s_i, a_i, r_i, s_i')$ in the mini-batch, the TD target $y_i$ is calculated using the target network:

$$y_i = r_i + \gamma \cdot \max_a Q(s_i', a; \theta^-) \tag{32}$$

Subsequently, the TD error $\delta_i$ for each transition is calculated with the online network:

$$\delta_i = y_i - Q(s_i, a_i; \theta) \tag{33}$$

ARTICLE IN PRESS

**Algorithm 1** DRL-based scheduling method for data-intensive tasks.

**Require:** Replay buffer capacity $N_{\mathcal{D}}$; learning rate $\alpha$; discount factor $\gamma$; mini-batch size $N_{\text{batch}}$; target update interval $N_{\text{target}}$; soft update factor $\tau$; exploration rate $\epsilon$ decay from $\epsilon_{\max}$ to $\epsilon_{\min}$

Initialize replay buffer $\mathcal{D}$ with capacity $N_{\mathcal{D}}$
Initialize exploration rate $\epsilon \leftarrow \epsilon_{\max}$
Initialize online network $Q(s, a; \theta)$ with random weights $\theta$
Initialize target network $Q(s, a; \theta^-)$ with weights $\theta^- \leftarrow \theta$
**for** episode $= 1$ to $N_{\text{episodes}}$ **do**
    Reset environment and observe initial state $s_0$
    **for** $t = 0, 1, 2, \ldots$ **do**
        Choose action $a_t$ with $\epsilon$-greedy policy: randomly select an action with probability $\epsilon$; otherwise $a_t = \text{argmax}_a Q(s_t, a; \theta)$
        Execute action $a_t$: schedule current task to the chosen server
        Observe reward $r_t$ and next state $s_{t+1}$
        Store transition $(s_t, a_t, r_t, s_{t+1})$ in replay buffer $\mathcal{D}$
        Anneal $\epsilon$ towards $\epsilon_{\min}$
        **if** $|\mathcal{D}| \geqslant N_{\text{batch}}$ **then**
            Sample random mini-batch $\mathcal{B} = \{(s_i, a_i, r_i, s_i')\}_{i=1}^{N_{\text{batch}}}$ from $\mathcal{D}$
            **for each** transition $(s_i, a_i, r_i, s_i') \in \mathcal{B}$ **do**
                Compute TD target: $y_i = r_i + \gamma \cdot \max_a Q(s_i', a; \theta^-)$
                Compute TD error: $\delta_i = y_i - Q(s_i, a_i; \theta)$
            **end for**
            Calculate loss: $L(\theta) = \frac{1}{N_{\text{batch}}} \sum_{i=1}^{N_{\text{batch}}} \frac{1}{2}\delta_i^2$
            Update $\theta$ by gradient descent: $\theta \leftarrow \theta - \alpha \cdot \nabla_\theta L(\theta)$
        **end if**
        **if** $t \equiv 0 \pmod{N_{\text{target}}}$ **then**
            Update target network: $\theta^- \leftarrow \tau \cdot \theta + (1 - \tau) \cdot \theta^-$
        **end if**
        **if** episode terminates at $t$ (end of task sequence) **then**
            **break**
        **end if**
    **end for**
**end for**

The DQN loss function $L(\theta)$ is then defined as the mean squared error of the TD errors over the mini-batch:

$$L(\theta) = \frac{1}{N_{\text{batch}}} \sum_{i=1}^{N_{\text{batch}}} \frac{1}{2}\delta_i^2 \tag{34}$$

Finally, the parameters $\theta$ of the online network are updated by performing a gradient descent step with learning rate $\alpha$ to minimize the loss:

$$\theta \leftarrow \theta - \alpha \cdot \nabla_\theta L(\theta) \tag{35}$$

ARTICLE IN PRESS

Between lines 22 and 24, the target network is periodically updated to keep it in sync with the online network. At every step $N_{\text{target}}$, the parameters $\theta^-$ of the target network are updated using a soft update mechanism:

$$\theta^- \leftarrow \tau \cdot \theta + (1 - \tau) \cdot \theta^- \tag{36}$$

# 5 Performance Evaluation

## 5.1 Experimental Setup

This section details the experimental setup for evaluating our DRL-based scheduler, including the workload and geo-distributed cloud settings, baseline methods, performance metrics, and the DQN training configurations.

### 5.1.1 Workload Configuration

The pattern of task arrival is modeled as a Poisson process, with exponentially distributed inter-arrival times. For each task $T_i$, we sample the baseline execution time $T_i^{\text{len}}$ and the input data size $T_i^{\text{data}}$ from a log-normal distribution $X \sim \text{LogNormal}(\mu, \sigma^2)$. This distribution guarantees strictly positive values that are reasonable for execution time and data size. Additionally, it captures the right-skewed, heavy-tailed characteristics commonly observed in real-world workloads [27, 28]. Given a target mean $m$ and coefficient of variation $v$, the distribution parameters are calculated as $\sigma = \sqrt{\ln(1 + v^2)}$ and $\mu = \ln(m) - \frac{1}{2}\sigma^2$.

To evaluate the robustness of our proposed method, we systematically vary three factors: (i) the task arrival rate (tasks per second), (ii) the mean baseline execution time $m_{\text{len}}$, and (iii) the mean input data size $m_{\text{data}}$, while holding the coefficients of variation constant at $v_{\text{len}} = v_{\text{data}} = 0.1$. By tuning these three parameters, we construct diverse workloads and test our method under conditions of high load, long task durations, and large data sizes to validate its effectiveness.

### 5.1.2 Geo-Distributed Cloud Settings

Our simulated cloud is made up of three different regions: `us-east-1-nyc-1`, `ap-southeast-1-bkk-1`, and `af-south-1-los-1`. Each region has eight heterogeneous servers based on different AWS instance types and capacities [29]. Table 2 lists the server configuration and its corresponding rental cost.

**Table 2** Server configuration and rental cost in each data center.

| Type | vCPU | Memory (GB) | Cost ($/hour) | Count |
|---|---|---|---|---|
| `c6g.medium` | 1 | 2 | 0.0456 | 2 |
| `c6g.large` | 2 | 4 | 0.0912 | 2 |
| `c6g.xlarge` | 4 | 8 | 0.1824 | 2 |
| `c6g.2xlarge` | 8 | 16 | 0.3648 | 2 |

ARTICLE IN PRESS

To model the geo-distributed operations correctly, we include network bandwidth and data transfer costs. We assume that bandwidth within a region is much higher than bandwidth across regions. Specifically, the intra-region bandwidth is 1000 MB/s, while the inter-region bandwidth is 10 MB/s. In addition, monetary costs are applied to data transfers according to AWS cloud pricing models [30]. Table 3 shows the matrix of data transfer costs used in our experiments.

**Table 3** The matrix of data transfer cost ($/GB).

| From \ To | us-east-1-nyc-1 | ap-southeast-1-bkk-1 | af-south-1-los-1 |
|---|---|---|---|
| us-east-1-nyc-1 | 0.01 | 0.08 | 0.02 |
| ap-southeast-1-bkk-1 | 0.09 | 0.08 | 0.09 |
| af-south-1-los-1 | 0.147 | 0.137 | 0.11 |

### 5.1.3 Baselines and Metrics

We compare the proposed task scheduling method with five baselines. The first three are popular real-time task scheduling methods: (i) **random** scheduler, each incoming task is assigned to a randomly chosen server; (ii) **round-robin** scheduler, each incoming task is assigned to a server in a cyclic order; (iii) **earliest** scheduler, the server with the least waiting time is always chosen. In addition, two recently proposed task scheduling methods for geo-distributed clouds are included: a **mathematical** optimization-based scheduler by Li *et al.* [12], and a **meta-heuristic** firefly algorithm-based scheduler by Ammari *et al.* [15]. Both approaches explicitly consider the geographical distribution of the data centers, and give realistic performance benchmarks for our learning-based scheduler.

We evaluate the proposed method and baselines using three metrics: average response time (ART), success rate (SR), and total operational cost (TOC). These metrics are derived from the multi-objective optimization problem formalized in Equation 18. ART measures the mean duration from task arrival to completion, including waiting time, data transmission time and execution time, thus reflecting the overall service performance of a scheduling method. SR captures the reliability of the scheduling policy under lease time constraints. TOC reflects the economic efficiency of the scheduling policy by accounting for both server rental and inter-region traffic charges.

### 5.1.4 DQN Training Settings

We build a Gymnasium [31] environment to simulate data-intensive task scheduling in a geo-distributed cloud, and train the DQN-based scheduler using the Tianshou RL library [32]. The Q-network is a fully connected DNN with four hidden layers of 256, 256, 128 and 64 neurons, respectively. We use the following training settings across all experiments: the learning rate is set to $\alpha = 5 \times 10^{-5}$; the target network is updated every $N_{\text{target}} = 500$ steps with a soft update factor of $\tau = 0.05$; the replay buffer size is $N_{\mathcal{D}} = 1{,}000{,}000$; and the batch size is $N_{\text{batch}} = 256$. We adopt an $\epsilon$-greedy exploration

ARTICLE IN PRESS

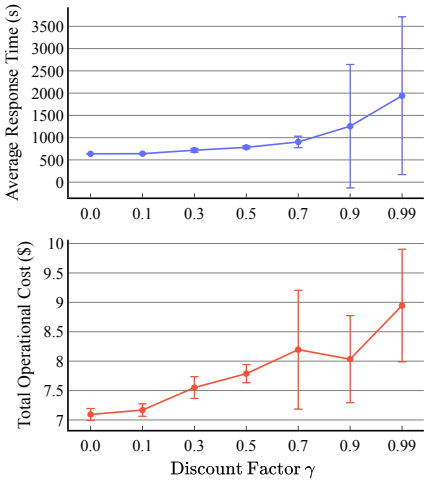


**Fig. 5** Sensitivity to the discount factor $\gamma$.

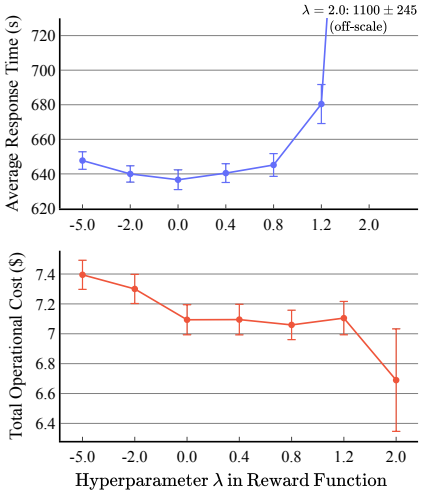


**Fig. 6** Sensitivity to the cost weight $\lambda$.

strategy, where the exploration rate decays linearly from $\epsilon_{\max} = 1.0$ to $\epsilon_{\min} = 0.05$ over the training process.

The discount factor $\gamma$ and the cost weight $\lambda$ in the reward function (Equation 31) directly affect both the performance stability of the DQN model and the optimization preference of the learned policy. To determine appropriate values for these two hyperparameters, we perform a grid search over $\gamma$ and $\lambda$. For each candidate value, we train a DQN model and evaluate it on the same workload for 100 independent runs, reporting the ART and TOC. We use the sample mean to measure expected performance and the sample standard deviation to quantify stability.

We first investigate the impact of the discount factor by testing $\gamma \in \{0.0, 0.1, 0.3, 0.5, 0.7, 0.9, 0.99\}$. As shown in Figure 5, when $\gamma \leqslant 0.5$, the models exhibit relatively small standard deviations, indicating stable performance. However, as $\gamma$ increases beyond this range, both ART and TOC become worse, and the instability becomes pronounced. Although $\gamma = 0.0$ achieves the best mean performance among all candidates, using a small but non-zero discount factor avoids an overly myopic objective and enables the agent to account for near-future effects within a short horizon. Therefore, we select $\gamma = 0.1$ for the remaining experiments.

We then evaluate the effect of the cost weight using $\lambda \in \{-5.0, -2.0, 0.0, 0.4, 0.8, 1.2, 2.0\}$, with results summarized in Figure 6. Overall, ART decreases as $\lambda$ increases from $-5.0$ to $0.0$, reaching its minimum at $\lambda = 0.0$ with a small standard deviation. In the same interval, TOC drops substantially, and then shows a clear plateau for $\lambda \in [0.0, 1.2]$. When $\lambda = 2.0$, the reward becomes dominated by cost, which can further reduce TOC, but at the expense of a significant increase in ART and noticeably unstable performance. Taken together, we choose $\lambda = 0.0$, as it yields the lowest ART among the candidates while still achieving near-minimal TOC.

ARTICLE IN PRESS

## 5.2 Experimental Results

In this section, we evaluate the proposed DRL-based scheduler from three aspects. We first measure its decision latency to verify the practicality of deploying a trained DQN model for real-time scheduling. We then compare its scheduling performance against the baselines under diverse workload conditions. Finally, we assess the generalization capability of the learned policy. For each workload configuration, we repeat the evaluation 100 times and report the mean of the performance metrics with the 95% confidence interval in the figures.

### 5.2.1 Scheduler Decision Latency

Before evaluating scheduling performance, we first measure the decision latency of our DRL-based scheduler to assess its practicality for real-time task scheduling. Decision latency is defined as the time taken by the scheduler to select a server for an incoming task. Specifically, it measures the elapsed time from feeding the current environment state into the trained DQN model to obtaining the output action. We evaluate this latency using two GPUs with different compute capabilities, NVIDIA RTX 5070Ti and NVIDIA RTX 4060, to examine the inference overhead of the trained DQN model. For each GPU, we record the latency over 100,000 scheduling decisions and summarize the minimum, mean, and maximum values in Table 4. The results show that the scheduler achieves sub-millisecond mean decision latency on both GPUs, with the maximum observed latency remaining below 2.5 ms. Overall, the inference overhead of the trained DQN model is small, indicating that decision latency is unlikely to be a major bottleneck for real-time scheduling in this study.

**Table 4** Decision latency of the DRL-based scheduler.

| GPU | Decision Latency (ms) | | |
|---|---|---|---|
| | Min | Mean | Max |
| NVIDIA RTX 5070Ti | 0.087 | 0.204 | 0.712 |
| NVIDIA RTX 4060 | 0.146 | 0.374 | 2.481 |

ARTICLE IN PRESS

### 5.2.2 Varying Arrival Rate

After confirming that our DQN scheduler is practical for real-time scheduling, we next evaluate its performance under different task arrival rates. Specifically, we test five rates: one task every 2 seconds, 1 tasks/s, 5 tasks/s, 10 tasks/s and 20 tasks/s, while fixing the mean baseline execution time $m_{\text{len}}$ = 1minute and the mean input data size $m_{\text{data}}$ = 100MB. As shown in Figure 7(a), the ART increases for all methods as the arrival rate grows from 0.5 to 20 tasks/s, due to intensified queuing and resource contention. Nevertheless, the proposed method consistently achieves the lowest ART across all load levels, indicating that it makes more effective scheduling decisions under both light and heavy loads.

ARTICLE IN PRESS

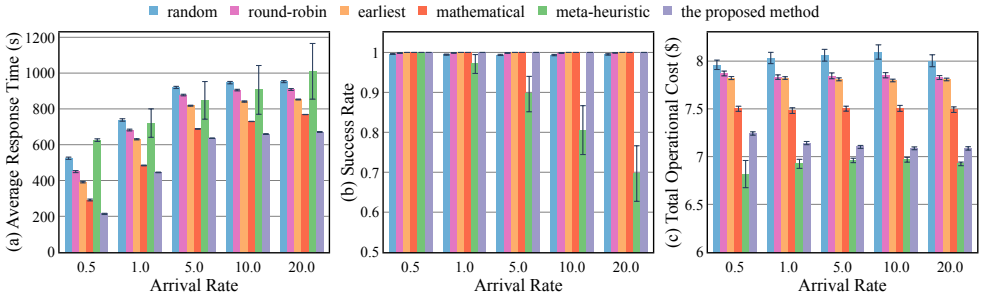


**Fig. 7** Performance under varying task arrival rates.

As illustrated in Figure 7(b), the proposed method achieves an SR of 100% across all arrival rates. In contrast, the SR of the meta-heuristic method decreases significantly as the load increase. Under high load conditions, the rise in task rescheduling frequency leads to a substantial increase in the response time of the meta-heuristic approach.

Figure 7(c) compares the TOC of all evaluated methods. The proposed method achieves the second-lowest TOC, while the meta-heuristic approach yields the lowest cost. This is mainly because the firefly-based meta-heuristic method explicitly treats operational cost as its optimization objective. It evaluates candidate schedules using a cost-centric objective function and iteratively drives the population toward lower-cost solutions via brightness and attractiveness updates. As a result, the method may favor already-leased servers even when their leases are about to expire, potentially triggering task rescheduling upon expiration. It may also prefer low-cost servers with limited capacity that cannot efficiently handle heavy tasks. In addition, it may assign tasks to lower-priced network regions even when the corresponding servers are heavily loaded. These behaviors increase queueing and rescheduling overhead, leading to the longer ART and lower SR observed in Figure 7(a) and 7(b). In contrast, our DQN-based scheduler jointly considers execution efficiency and operational cost in the reward function for multi-objective optimization. Although its TOC is 1.7%–6.2% higher than the meta-heuristic method, it reduces the ART by 25%–66%, resulting in a more balanced scheduling performance.

### 5.2.3 Varying Baseline Execution Time

We then evaluate our method and the baselines under different task execution durations. In this experiment, we fix the arrival rate at 5 tasks/s and set the mean input data size to $m_{\text{data}} = 100\text{MB}$. We vary the mean baseline execution time as $m_{\text{len}} \in \{1\text{s}, 10\text{s}, 1\text{min}, 5\text{min}, 10\text{min}\}$. As shown in Figure 8(a), the proposed method has the lowest ART for both long and short tasks. The meta-heuristic method performs very poorly on short tasks, while on the other hand the mathematical method experiences a performance degradation with long tasks. In contrast, the proposed DRL-based scheduler is robust for all tested task lengths, always achieving the shortest response times.

ARTICLE IN PRESS

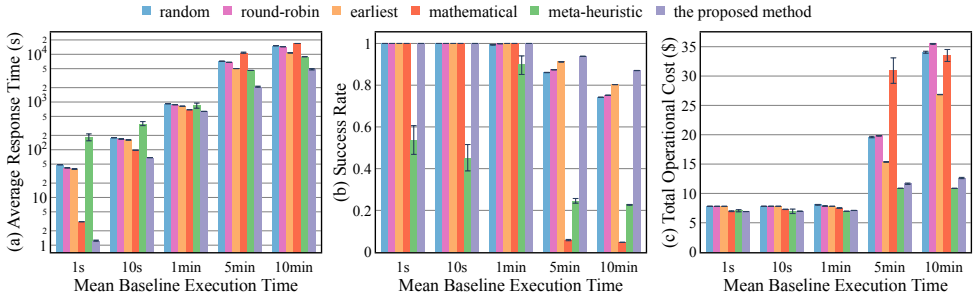


**Fig. 8** Performance under varying baseline task execution time.

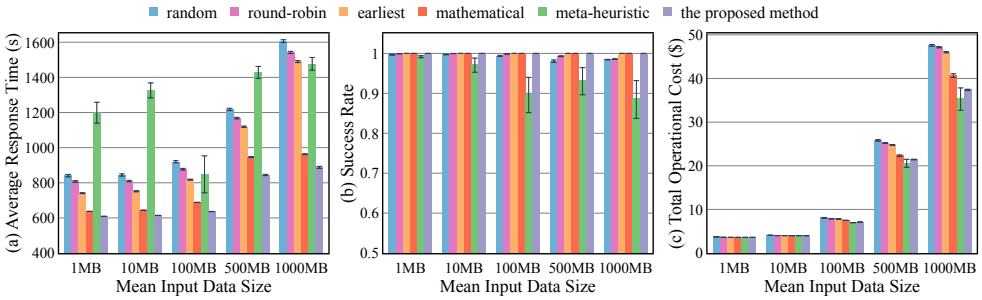


**Fig. 9** Performance under varying input data sizes required by tasks.

Figure 8(b) shows how the SR of all methods changes as the task length increases. When $m_{\text{len}} \in \{1\text{s}, 10\text{s}, 1\text{min}\}$, the proposed method maintains an SR of 100%. However, when $m_{\text{len}} \in \{5\text{min}, 10\text{min}\}$, the SR drops for all methods. This is mainly because we fix the lease duration parameter $\tau_{\text{rent}}$ in Equation 6 to 1 hour, which models a leasing granularity (e.g., a minimum billing unit) and allows us to study scheduling performance under a lease-time constraint. As tasks run longer on a server, they are more likely to exceed the remaining lease term and be interrupted by server expiration. In this regime, the baselines do not have explicit mechanisms to account for tasks that are nearing server lease expiration, resulting in lower SR. By contrast, the higher SR of our approach indicates that it can more effectively utilize the lease period by completing tasks within the remaining server lease term.

Figure 8(c) shows a comparison of the total operational costs. The meta-heuristic approach attains the lowest cost in most cases; our proposed method follows closely, with costs at most 16% higher. Overall, the proposed method consistently outperforms all five baselines, delivering lower response times and higher success rates while keeping the operational cost competitive. These results indicate that it remains economical and reliable under varying task execution durations.

ARTICLE IN PRESS

ARTICLE IN PRESS

**Fig. 10** Adaptability of the DQN-based scheduler under different workload configurations.

### 5.2.4 Varying Input Data Size

We next vary the mean input data size to evaluate the scheduling performance under different data sizes. Specifically, the mean data size varies as $m_{\text{data}} \in \{1\text{MB}, 10\text{MB}, 100\text{MB}, 500\text{MB}, 1000\text{MB}\}$, while the arrival rate is fixed at 5 tasks/s and the mean baseline execution time is set to $m_{\text{len}} = 1\text{min}$. As illustrated in Figure 9(a), ART generally increases as the data sizes grow, due to increased data transmission times. The proposed method, however, has the lowest response time for every tested data size. By considering the bandwidth limitations of inter-regions, it intelligently schedules data-intensive tasks to reduce data transmission latency. In contrast, random, round-robin and earliest schedulers show significant increases in ART with growing data size, as they do not take network bandwidth limitations into account and tend to transfer large data volumes across regions on low-speed WAN links.

Figure 9(b) shows the SR under different mean input data sizes. The proposed method achieves an SR of 100% in the evaluated data-size settings, whereas the random and round-robin exhibit a decreasing SR as the data size increases. This degradation occurs because these baselines ignore bandwidth bottlenecks when handling large cross-region data transfers. As a result, data transmission consumes a large fraction of the lease period, leaving insufficient time for task execution and eventually causing task failures.

The cost results under varying input data sizes are summarized in Figure 9(c). When the data size is small (up to 100 MB), all methods incur similar costs because data transfer is negligible and server rental dominates. As the data size increases (500 MB and above), the cost differences become more pronounced: the meta-heuristic method achieves the lowest cost, while the proposed approach incurs only a slightly higher cost but delivers substantially better ART and SR. Overall, by accounting for both transmission latency and cross-region traffic charges, the proposed scheduler achieves consistently strong performance with cost close to the meta-heuristic method under the tested data sizes.

ARTICLE IN PRESS

ARTICLE IN PRESS

### 5.2.5 Adaptability to Different Workload Configurations

To further examine adaptability, we evaluate a DQN-based scheduler under different workload configurations. Specifically, we select the DQN model trained under the mid-range workload used in the previous experiments, where the arrival rate is 5 tasks/s, $m_{\text{len}}$ = 1minute, and $m_{\text{data}}$ = 100MB. We then test this model under three heavier workload settings while keeping all other parameters unchanged: (A) a higher arrival rate of 20 tasks/s, (B) longer tasks with $m_{\text{len}}$ = 10minutes, and (C) larger data volume with $m_{\text{data}}$ = 1000MB.

After evaluating the mid-range model directly in these target scenarios, we further adapt it by continuing training within each target workload setting to obtain an updated model. Figure 10 summarizes the performance of these models in terms of ART, SR, and TOC, where **scratch** represents a DQN model trained from random initialization under the current target workload, **zero-shot** represents the mid-range DQN model evaluated directly under the target workload without additional training, and **fine-tuned** represents the DQN model obtained by warm-starting from the mid-range model and continuing training under the target workload.

As shown in Figure 10, the mid-range model transfers well when the arrival rate increases, achieving performance close to the scratch model across all three metrics. However, when facing longer tasks or larger data volumes, the zero-shot model's performance degrades significantly, especially in terms of ART and TOC. This is because the mid-range model has not encountered such extreme conditions during training, leading to suboptimal scheduling decisions. Encouragingly, continuing training within the target workload can effectively adapt the mid-range model to the new conditions. The fine-tuned model achieves performance that is comparable to scratch in our experiments. This adaptability is important for real-world geo-distributed cloud environments, where workload characteristics can vary widely over time.

## 6 Conclusion

This paper proposes a DRL-based approach for real-time scheduling of data-intensive tasks in geo-distributed cloud environments. We first formulate the scheduling problem as a multi-objective optimization problem that aims to minimize average response time, maximize task success rate, and reduce total operational cost. To enable learning-based decision-making, we further model the task scheduling process as a MDP. On this basis, a DQN agent is trained to make scheduling decisions by dynamically tracking server states while accounting for inter-regional bandwidth limitations and data transfer costs. Experimental results show that our method adapts well across the diverse workload configurations considered in this study. As future work, we plan to extend this framework to a multi-agent design [33] that jointly optimizes task scheduling and server leasing decisions, enabling flexible choices of server lease duration to further improve task execution efficiency and reduce operational cost in large-scale geo-distributed cloud environments. Meanwhile, we will explore large language model-assisted scheduling [34] to enhance generalization and interpretability.

ARTICLE IN PRESS

## References


[1] Zhao, Z., Chen, Z., Lin, Z., Zhu, W., Qiu, K., You, C., Gao, Y.: LEO Satellite Networks Assisted Geo-Distributed Data Processing. IEEE Wireless Communications Letters **14**(2), 405–409 (2025)

[2] Gouasmi, T., Kacem, A.H.: Global reduction for geo-distributed MapReduce across cloud federation. Future Generation Computer Systems **162**, 107492 (2025)

[3] Li, Z., Feng, W., Cai, W., Yu, H., Luo, L., Sun, G., Du, H., Niyato, D.: Accelerating Geo-Distributed Machine Learning With Network-Aware Adaptive Tree and Auxiliary Route. IEEE/ACM Transactions on Networking **32**(5), 4238–4253 (2024)

[4] Cai, X., Li, M., Zhang, Y., Zhao, T., Zhang, W., Chen, J.: Multitasking bi-level evolutionary algorithm for data-intensive scientific workflows on clouds. Expert Systems with Applications **238**, 121833 (2024)

[5] Hogade, N., Pasricha, S.: Game-Theoretic Deep Reinforcement Learning to Minimize Carbon Emissions and Energy Costs for AI Inference Workloads in Geo-Distributed Data Centers. IEEE Transactions on Sustainable Computing **10**(4), 628–641 (2025)

[6] Wu, Y., Tang, S., Yu, C., Yang, B., Sun, C., Xiao, J., Wu, H., Feng, J.: Task Scheduling in Geo-Distributed Computing: A Survey. IEEE Transactions on Parallel and Distributed Systems, 1–14 (2025)

[7] Wang, B., Wang, J., Sun, Y.-E., Huang, H.: Copo: Joint Cost and Performance Optimization for Task Placement in Geo-Distributed Clouds. In: 2025 IEEE 33rd International Conference on Network Protocols (ICNP), pp. 1–12 (2025)

[8] Khalid, S., Ahmad, I.: Dual Optimization of Revenue and Expense in Geo-Distributed Data Centers Using Smart Grid. IEEE Transactions on Cloud Computing **11**(2), 1622–1635 (2023)

[9] Dai, S., Li, S., Tang, H., Ning, X., Fang, F., Fu, Y., Wang, Q., Cheng, L.: MARP: A Cooperative Multiagent DRL System for Connected Autonomous Vehicle Platooning. IEEE Internet of Things Journal **11**(20), 32454–32463 (2024)

[10] Gu, Y., Liu, Z., Dai, S., Liu, C., Wang, Y., Wang, S., Theodoropoulos, G., Cheng, L.: Deep Reinforcement Learning for Job Scheduling and Resource Management in Cloud Computing: An Algorithm-Level Review (2025) arXiv:2501.01007

[11] Yuan, H., Bi, J., Zhou, M.: Geography-Aware Task Scheduling for Profit Maximization in Distributed Green Data Centers. IEEE Transactions on Cloud Computing **10**(3), 1864–1874 (2022)


ARTICLE IN PRESS

ARTICLE IN PRESS

[12] Li, X., Chen, F., Ruiz, R., Zhu, J.: MapReduce Task Scheduling in Heterogeneous Geo-Distributed Data Centers. IEEE Transactions on Services Computing **15**(6), 3317–3329 (2022)

[13] Luo, L., Zhao, G., Xu, H., Yu, Z., Xie, L.: Achieving Cost Optimization for Tenant Task Placement in Geo-Distributed Clouds. IEEE/ACM Transactions on Networking **32**(2), 1391–1406 (2024)

[14] Yuan, H., Bi, J., Zhou, M., Liu, Q., Ammari, A.C.: Biobjective Task Scheduling for Distributed Green Data Centers. IEEE Transactions on Automation Science and Engineering **18**(2), 731–742 (2021)

[15] Ammari, A.C., Labidi, W., Mnif, F., Yuan, H., Zhou, M., Sarrab, M.: Firefly algorithm and learning-based geographical task scheduling for operational cost minimization in distributed green data centers. Neurocomputing **490**, 146–162 (2022)

[16] Xie, T., Li, C., Hao, N., Luo, Y.: Multi-objective optimization of data deployment and scheduling based on the minimum cost in geo-distributed cloud. Computer Communications **185**, 142–158 (2022)

[17] Cheng, L., Wang, Y., Cheng, F., Liu, C., Zhao, Z., Wang, Y.: A Deep Reinforcement Learning-Based Preemptive Approach for Cost-Aware Cloud Job Scheduling. IEEE Transactions on Sustainable Computing **9**(3), 422–432 (2024)

[18] Chen, X., Yang, L., Chen, Z., Min, G., Zheng, X., Rong, C.: Resource Allocation With Workload-Time Windows for Cloud-Based Software Services: A Deep Reinforcement Learning Approach. IEEE Transactions on Cloud Computing **11**(2), 1871–1885 (2023)

[19] Zhang, S., Xu, M., Bryan Lim, W.Y., Niyato, D.: Sustainable AIGC Workload Scheduling of Geo-Distributed Data Centers: A Multi-Agent Reinforcement Learning Approach. In: GLOBECOM 2023 - 2023 IEEE Global Communications Conference, pp. 3500–3505 (2023)

[20] Downey, A.B.: A parallel workload model and its implications for processor allocation. Cluster Computing **1**(1), 133–145 (1998)

[21] Eager, D.L., Zahorjan, J., Lazowska, E.D.: Speedup versus efficiency in parallel systems. IEEE Transactions on Computers **38**(3), 408–423 (1989)

[22] Dai, S., Saurabh, N., Wang, Q., Nian, J., Kan, S., Mao, Y., Cheng, L.: Quantum Reinforcement Learning for QoS-Aware Real-Time Job Scheduling in Cloud Systems. IEEE Systems Journal **19**(2), 471–482 (2025)

[23] Clifton, J., Laber, E.: Q-Learning: Theory and Applications. Annual Review of Statistics and Its Application **7**(1), 279–301 (2020)

ARTICLE IN PRESS

ARTICLE IN PRESS

[24] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., Hassabis, D.: Human-level control through deep reinforcement learning. Nature **518**(7540), 529–533 (2015)

[25] Yan, H., Gu, Y., He, H., Ning, X., Wang, Q., Cheng, L.: DNN-Based Task Partitioning and Offloading in Edge-Cloud Collaboration Within Electric Vehicles. IEEE Transactions on Consumer Electronics **71**(2), 4100–4109 (2025)

[26] Zhao, X., Chen, P., Tang, L.C.: Condition-based maintenance via Markov decision processes: A review. Frontiers of Engineering Management **12**(2), 330–342 (2025)

[27] Marciani, M.G., Castrillo, M., Utrera, G., Acosta, M.C., Kinoshita, B.P., Doblas-Reyes, F.: Evaluating the impact of task aggregation in workflows with shared resource environments: Use case for the MONARCH application. Geoscientific Model Development **18**(23), 9709–9721 (2025)

[28] Fingler, H., Tarte, I., Yu, H., Szekely, A., Hu, B., Akella, A., Rossbach, C.J.: Towards a Machine Learning-Assisted Kernel with LAKE. In: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, pp. 846–861 (2023)

[29] Amazon.Com: Cloud Compute Instances – Amazon EC2 Instance Types – AWS (2025). https://aws.amazon.com/ec2/instance-types/ Accessed 2025-10-15

[30] Amazon.Com: EC2 On-Demand Instance Pricing (2025). https://aws.amazon.com/ec2/pricing/on-demand/ Accessed 2025-10-15

[31] Towers, M., Kwiatkowski, A., Terry, J., Balis, J.U., Cola, G.D., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., Perez-Vicente, R., Pierré, A., Schulhoff, S., Tai, J.J., Tan, H., Younis, O.G.: Gymnasium: A Standard Interface for Reinforcement Learning Environments (2024) arXiv:2407.17032

[32] Weng, J., Chen, H., Yan, D., You, K., Duburcq, A., Zhang, M., Su, Y., Su, H., Zhu, J.: Tianshou: A Highly Modularized Deep Reinforcement Learning Library. Journal of Machine Learning Research **23**(267), 1–6 (2022)

[33] Cheng, L., He, H., Gu, Y., Liu, Q., Zhao, Z., Fang, F.: MARS: Multi-Agent Deep Reinforcement Learning for Real-Time Workflow Scheduling in Hybrid Clouds with Privacy Protection. In: 2024 IEEE 30th International Conference on Parallel and Distributed Systems (ICPADS), pp. 657–666 (2024)

[34] Pei, H., Gu, Y., Sun, Y., Wang, Q., Liu, C., Chen, X., Cheng, L.: LLM-based cost-aware task scheduling for cloud computing systems. Journal of Cloud Computing **14**(1), 81 (2025)

ARTICLE IN PRESS